

Chamadas de API do Codex no Coding_Machine

Pergunta: quantas chamadas de API o motor autônomo gastou por pacote do backlog, em que tentativa cada uma aconteceu e com que modelo. **Fonte:** `.autodev/state.db`, tabela `attempts` (o próprio motor grava uma linha por invocação). **Janela:** 28/09/2026 14:19 a 29/09/2026 08:51 — **sprints 003 e 004**, as únicas no protocolo de registro de hoje (aviso 2). **Data do relatório:** 29/09/2026. **Total no período: 50 chamadas do Codex**, 11 delas aprovadas (revisão + integração).

1. Estratégias de alocação de modelo

Esta é a seção que explica o resto do documento: o que se mede aqui é consequência de **como o modelo foi escolhido em cada tentativa**.

1.1 Padrão de esforço (fixado pelo teste A/B)

O teste A/B de custo mediu as combinações de modelo × esforço e o resultado foi direto: **o DeepSeek sai mais barato no `high` e o Codex no `low`**. Por chamada, o `deepseek-flash/high` custou **US\$ 0,0027** contra **US\$ 0,0066** do `sol/low`, **US\$ 0,0105** do `luna/low`, **US\$ 0,0133** do `pro/high` e **US\$ 0,0201** do `astra/low` (prévia medida, 12 braços). **Todos os testes daqui em diante seguem esse padrão:** DeepSeek sempre `high`, Codex sempre `low` — inclusive o revisor, que passou a rodar `deepseek-flash/high` em 29/09.

1.2 A última estratégia de escalada (degraus 1 → 5) — encerrada em 29/09

Escalada por tentativa, um degrau por chamada, com o modelo vindo do **contador da task**: 1ª `luna/low` → 2ª `terra/low` → 3ª `sol/low` → 4ª `astra/low` → 5ª+ `astra/low` (o 4º degrau era `sol/medium` até a P-13, em 29/09 — era o único degrau que escalava por ESFORÇO).

- **Janela de uso:** 28/09 14:19 a 29/09 08:51 (as chamadas no escopo deste relatório).
- **Pacotes que fecharam nela (11):** 003/P01, 003/P02, 003/P03, 003/P04, 003/P05, 003/P06, 003/P07, 004/P01, 004/P02, 004/P03, 004/P04.
- **Abertos quando a estratégia foi trocada (0):**
 - nenhum
- **Encerrada porque:** o autor decidiu que a escada passa a escalar **só por modelo** (P-13) e, em seguida, definiu uma estratégia nova (1.3), depois de o motor gastar a escada inteira em pacotes cujo defeito era de contrato/plano nosso, não de capacidade do modelo.

1.3 A estratégia nova — “3 degraus” (vigente desde 29/09/2026)

Três degraus, com **parada obrigatória para decisão do autor**:

1º `luna/low` → 2º `astra/low` → 3º `luna/low` — e, se o 3º não resolver, o motor **não insiste nem bloqueia**: para em `WAITING_HUMAN`, avisa no Telegram e espera resposta (*repetir os três degraus, mudar de estratégia ou encerrar*).

- **Regra de convivência:** cada task grava a estratégia com que **começou**. Trocar a política não muda a escada de quem já entrou — os pacotes em andamento terminam na escada antiga (é por isso que os dois blocos convivem neste relatório).
- **Ainda sem pacote executado:** nenhum pacote da 003 ou da 004 rodou nos “3 degraus”. O 003/P07 chegou a ser marcado nela (foi o primeiro task novo depois da decisão), mas **fechou na escada antiga** — a troca de código só passou a valer para o processo seguinte. **O próximo pacote novo** (próxima sprint) será o primeiro a rodar a estratégia: 1º `luna/low`, 2º `astra/low`, 3º `luna/low` e, falhando o terceiro, a parada para decisão do autor.
- **Modelos antigos continuam demarcados** nos gráficos (laranja): `sol/med*` foi a matriz até 27/09 e aparece nas chamadas das 003/004 que rodaram antes da troca.

Avisos

1. **Chamada é custo — com a procedência declarada.** Cada linha conta **uma invocação** do agente. O motor passou a **gravar tokens** em 28/09 (P-10, lendo o rodapé do agente): das 50 chamadas no escopo, **29 têm token medido** e as outras foram **estimadas** pela régua do modelo. Todo valor em US\$ diz de qual dos dois vem — sólido é medição, hachurado é estimativa.
2. **Por que só as sprints 003 e 004.** As sprints **001 e 002 estão fora deste relatório**, e o motivo é de registro, não de mérito: a **001** é história reconstruída (14 dos 15 pacotes são linhas retroativas, sem chamada de API nenhuma), e a **002** foi levantada depois do fato, com várias aprovações por pacote (até 6 no mesmo pacote) e sem um único token medido. Nas sprints **003 e 004** o protocolo é o de hoje: uma chamada de API por tentativa, revisão registrada e token lido do rodapé do agente. Comparar as quatro juntas mediria a diferença de protocolo, não a de modelo.
3. **A 003 fechou.** Entrou em 28/09 22:58 e fecha com **7 de 7 pacotes integrados** — o último (P07) em 5 chamadas, aprovado no `astra/low`. Os números dela **não mudam mais**; o que ainda vai mudar é a sprint seguinte.
4. **A sprint 004 está fechada** (4/4 integradas em 28/09) — os números dela não mudam mais.
5. **“Nª tentativa” não é o degrau da escada de modelos — são dois contadores.** O número nas tabelas é a **chamada** (`attempt`, sequência do banco, sempre `max+1`); o modelo vem do **contador da task** (`tentativas`), pelo mapa `1ª→luna/low ... 5ª→astra/low`. Quando o contador é reiniciado (rearme por dependência integrada, reabertura por defeito de contrato) ele **volta ao degrau barato** enquanto a numeração da chamada continua — é por isso que existe `luna/low` numa 7ª chamada. O degrau mede “quantas vezes chamou”, não “quão forte era o modelo”. Nas falhas de infraestrutura (`CODEX_QUOTA`, `NETWORK_ERROR`, `ENVIRONMENT_ERROR` ...), a política declara `classes_sem_escalamento` — mas **essa lista não chega à escolha do modelo**: o orquestrador usa o mapa do contador e ignora o tier da decisão. Na prática, espera de cota escalona como qualquer falha. É um defeito de fiação, não uma intenção.
6. **A partir da 5ª chamada o mapa satura no topo** (`min(tentativa, 5) → astra/low`): degraus 5 a 15 repetem o mesmo modelo. No período isso **não** virou desperdício: são 8 chamadas no topo (16.0%) contra 13 no degrau mais barato (26.0%) — a cauda é curta porque a maioria dos pacotes aprovou antes do 5º degrau (tabela 3).

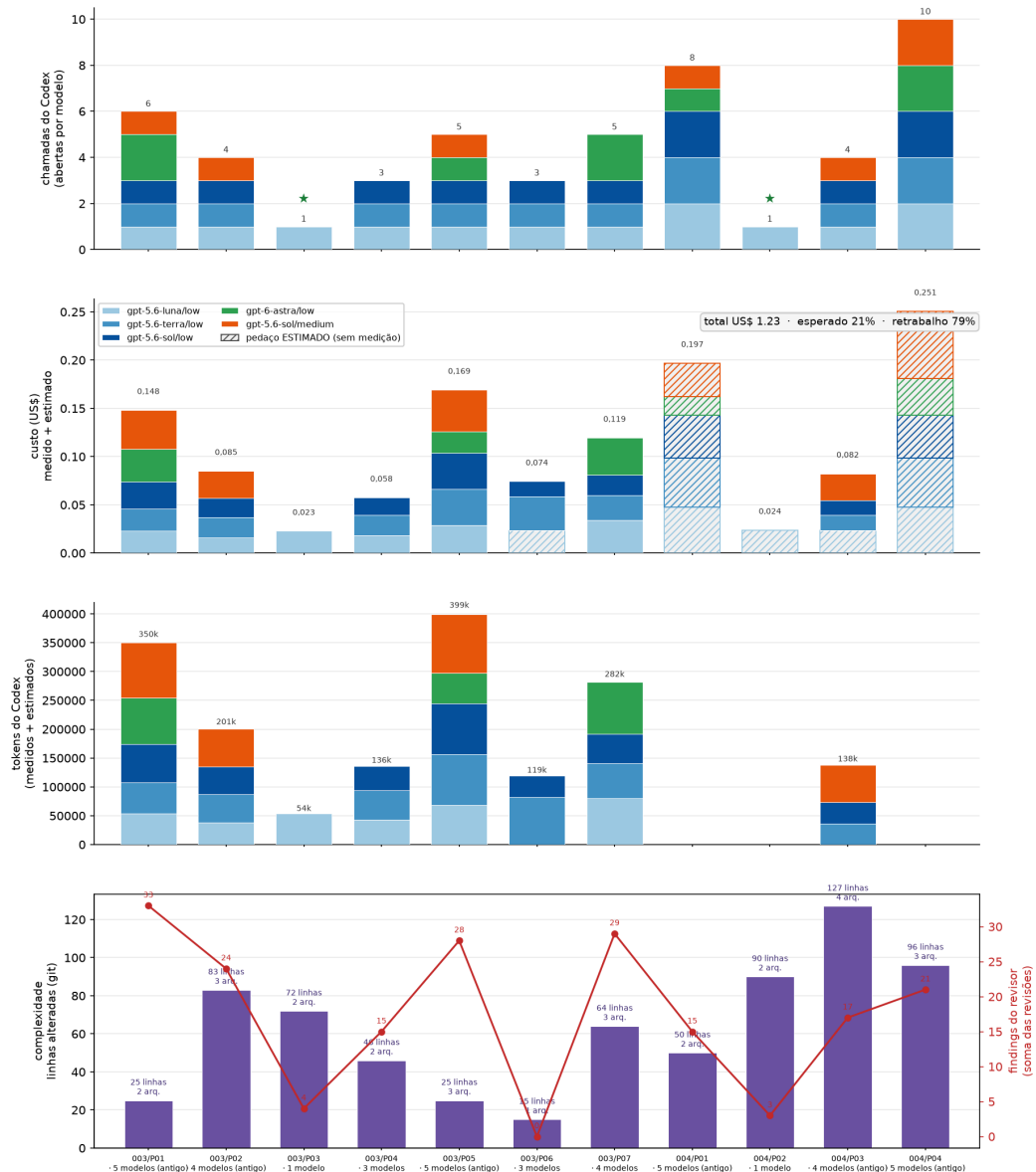
7. **0 combinação(ões) fora da escada declarada:** nenhuma. As combinações `.../medium` são o **modelo antigo**: `sol/medium` ocupou o 4º degrau até 27/09 (e continuou sendo usado por quem já estava no meio do caminho, até a P-13 de 29/09). Não é desvio de protocolo — é a matriz aposentada, demarcada em **laranja** nos gráficos para o autor ver quanto rodou no modelo velho.
8. **A numeração por pacote tem buracos.** Rearme por dependência integrada e reabertura por defeito de contrato removem/renomeiam tentativas, então 1 pacote(s) (P01 (sprint 004)) têm sequência descontínua — marcados com ▲ na tabela 1. Consequência para a leitura: o degrau 1 pode se repetir na história de um mesmo pacote, e “15 primeiras tentativas para 15 pacotes” é coincidência, não regra.

O que os números dizem

- **50 chamadas do Codex** em 11 pacotes com execução registrada. A mediana é de 4 chamadas por pacote e a média 4.5.
- **22.0% das chamadas são de 1ª tentativa** e 40.0% acontecem até a 2ª. Metade do gasto (3ª tentativa em diante) está na cauda: são poucos pacotes que consumiram a escada inteira.
- **2 pacote(s) resolveram com uma única chamada:** P03 (sprint 003), P02 (sprint 004).
- **Os campeões de gasto:** P04 (sprint 004, 10 chamadas), P01 (sprint 004, 8 chamadas), P01 (sprint 003, 6 chamadas). Os dois são casos conhecidos: a P02 da sprint 2 pedia reescrita de contrato de teste (o mesmo arquivo para tasks diferentes) e a P04 da sprint 4 gastou a escada consertando o próprio comando de teste — 10 tentativas, das quais o defeito era meu, não do agente (decisão D-23).
- **A cauda direita do gráfico 1 é o sintoma mais caro do período:** 5 chamadas em degraus 11 a 15, todas em pacotes que só destravaram quando o **defeito de motor** foi corrigido (D-19, D-24, D-25) — nenhuma delas é “o modelo errado tentando mais”.
- **10 das 50 chamadas (20.0%) foram gastas por defeito do nosso teste/plano**, e 2 (4.0%) por infraestrutura (cota/crash). Descontadas, sobram **38 chamadas (76.0%)** atribuíveis ao trabalho do modelo — o denominador honesto para comparar modelos (seção “Descontando”).
- **Onde a escada se paga (tabela 3):** 4 pacote(s) aprovaram até a 3ª chamada; 4 na 4ª–5ª; 3 da 6ª em diante. O degrau caro (`astra/low`) assinou 4 aprovação(ões) — sempre em pacote que carregava, junto, defeito de contrato nosso.

Gráfico 0 — os quatro painéis, no mesmo eixo x

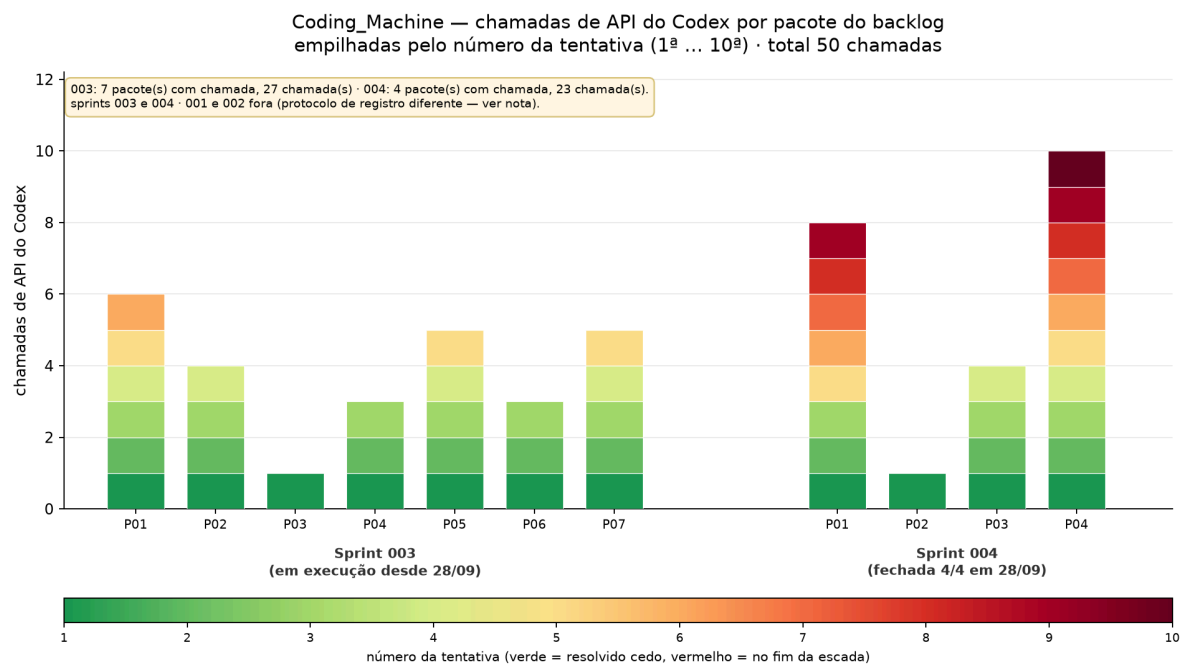
Coding_Machine — os quatro gráficos do mesmo eixo: cada pacote, aberto por modelo de tentativa
sprints 003 e 004 · 001 e 002 fora (protocolo de registro diferente — ver nota)
★ = pacote fechado do início ao fim com UM modelo só · laranja = modelo de matriz antiga (fora da escada vigente) · hachurado = pedaço estimado (sem medição)



Quatro painéis empilhados por pacote, abertos por modelo de tentativa (chamadas, custo, tokens e complexidade)

Os quatro na mesma sequência de pacotes, para comparar sem trocar de imagem: **chamadas** (1), **custo em US\$** (2), **tokens** (3) e **complexidade** (4 — linhas e arquivos do commit de fechamento, do git, mais a linha vermelha com os findings do revisor). Cor = par modelo/esforço; **laranja é modelo de matriz antiga** (fora da escada vigente); hachurado = pedaço **estimado** por não ter medição (P-10); ★ = pacote fechado do início ao fim com um **único** modelo. O detalhe de cada um vem nas seções seguintes.

Gráfico 1 — chamadas por pacote, empilhadas pela tentativa



Chamadas de API do Codex por pacote do backlog, empilhadas pelo número da tentativa

Cada coluna é um pacote; a altura é quantas vezes o Codex foi chamado nele; a cor diz **em que altura da escada** a chamada aconteceu (verde = cedo, vermelho = fim da escada). Total: 50 chamadas.

Tabela 1 — por pacote

chamadas é o custo; A/R/S são as **avaliações do modelo aprovador** naquele pacote (aprovadas / reprovadas / chamadas que nem chegaram a ser avaliadas); 1ª-última mostra a faixa de chamadas do pacote; aprovada na diz **em que chamada** (e com que modelo) a aprovação saiu; infra e culpa teste/plano separam o que **não era do modelo** (cota/crash e defeito de teste/plano, atribuição curada descrita abaixo); do modelo é o que sobra.

retrab. bruto = **reprovações do aprovador ÷ aprovações do aprovador**, em **múltiplo**: 2x significa duas reprovações para cada aprovação entregue (não é porcentagem de nada — pode passar de 1x, e é por isso que vai em x e não em %); retrab. ajust. desconta as reprovações que foram culpa do **nosso teste/plano** — nunca as do codificador. Pacote sem aprovação nenhuma fica —. No total: **25 reprovações ÷ 11 aprovações = 2,3x bruto e 2,3x ajustado** (a escada cobrou 0 reprovações que eram defeito do NOSSO teste/plano). Se o denominador for *chamadas avaliadas* em vez de aprovações — $reprov \div (aprov+reprov)$, aí sim uma fatia — os mesmos números ficam 69.4% e 69.4%.

As três últimas colunas são para **auditoria**: início-fim é a janela de data-hora das chamadas do pacote (fuso local, 28/09 22:58→23:46), e linhas/arq. é o tamanho que o pacote realmente entregou — contado no commit de fechamento com `git show --numstat` (a mesma fonte da complexidade no painel 4). Na coluna de modelos, o nome vai encurtado (luna/low = gpt-5.6-luna/low ; sol/med* = matriz aposentada em 27/09).

sprint	pacote	cham.	A/R/S	1ª- última	aprovada na	infra	culpa teste/plano	do modelo	retrab. bruto	retrab. ajust.	modelos usados	início- fim	linhas/arq
003	P01	6	1/5/0	1ª-6ª	6ª (gpt-6-astra/low)	0	0	6	5x	5x	luna/low, sol/low, sol/med* , terra/low, astra/low	28/09 22:58 → 23:46	25/2
003	P02	4	1/3/0	1ª-4ª	4ª (gpt-5.6-sol/medium)	0	0	4	3x	3x	luna/low, sol/low, sol/med* , terra/low	28/09 23:56 → 29/09 00:17	83/3
003	P03	1	1/0/0	1ª-1ª	1ª (gpt-5.6-luna/low)	0	0	1	0x	0x	luna/low	29/09 00:28 → 00:28	72/2
003	P04	3	1/2/0	1ª-3ª	3ª (gpt-5.6-sol/low)	0	0	3	2x	2x	luna/low, sol/low, terra/low	29/09 00:34 → 00:48	46/2
003	P05	5	1/4/0	1ª-5ª	5ª (gpt-6-astra/low)	0	0	5	4x	4x	luna/low, sol/low, sol/med* , terra/low, astra/low	29/09 00:55 → 01:40	25/3
003	P06	3	1/0/2	1ª-3ª	3ª (gpt-5.6-sol/low)	1	0	2	0x	0x	luna/low, sol/low, terra/low	29/09 01:50 → 08:01	15/1
003	P07	5	1/4/0	1ª-5ª	5ª (gpt-6-astra/low)	0	0	5	4x	4x	luna/low, sol/low, terra/low, astra/low	29/09 08:14 → 08:51	64/3
004	P01	8	1/2/5	1ª-9ª ▲	8ª (gpt-5.6-sol/low)	0	5	3	2x	2x	luna/low, sol/low, sol/med* , terra/low, astra/low	28/09 14:19 → 14:54	50/2
004	P02	1	1/0/0	1ª-1ª	1ª (gpt-5.6-luna/low)	0	0	1	0x	0x	luna/low	28/09 15:30 → 15:30	90/2
004	P03	4	1/2/1	1ª-4ª	4ª (gpt-5.6-sol/medium)	1	0	3	2x	2x	luna/low, sol/low, sol/med* , terra/low	28/09 15:37 → 19:43	127/4

sprint	pacote	cham.	A/R/S	1ª– última	aprovada na	infra	culpa teste/plano	do modelo	retrab. bruto	retrab. ajust.	modelos usados	início– fim	linhas/arq
004	P04	10	1/3/6	1ª– 10ª	10ª (gpt- 6- astra/low)	0	5	5	3x	3x	luna/low, sol/low, sol/med* , terra/low, astra/low	28/09 14:30 → 15:17	96/3

Tabela 3 — em que chamada a aprovação veio

O valor marginal da escada: onde os pacotes **efetivamente** destravaram. É esta tabela que decide se a 4ª/5ª posição da escada se paga.

aprovada na	pacotes	quais	modelo que aprovou
1ª chamada	2	P03 (003), P02 (004)	gpt-5.6-luna/low
2ª–3ª	2	P04 (003), P06 (003)	gpt-5.6-sol/low
4ª–5ª	4	P02 (003), P05 (003), P07 (003), P03 (004)	gpt-5.6-sol/medium, gpt-6-astra/low
6ª–10ª	3	P01 (003), P01 (004), P04 (004)	gpt-5.6-sol/low, gpt-6-astra/low
11ª–15ª	0	—	—

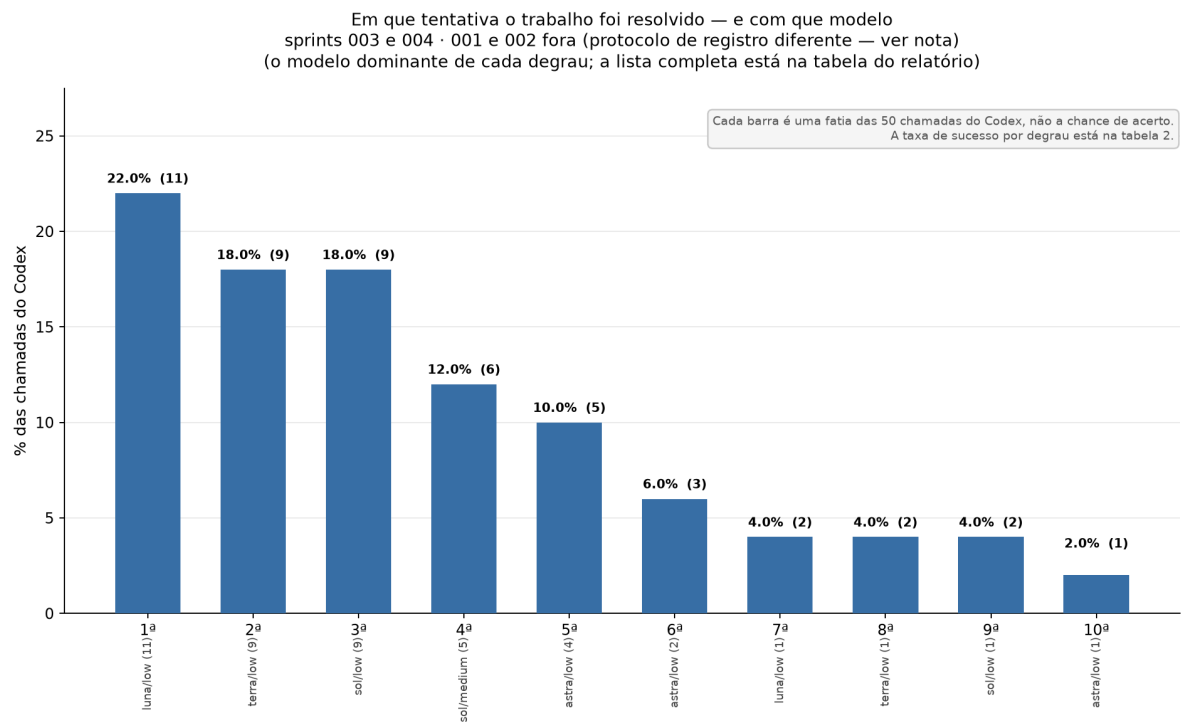
Descontando o que não era do modelo

A classe da falha o motor grava; **de quem era a culpa, não**. As janelas abaixo foram atribuídas à mão, olhando os logs e as decisões — e por isso aparecem em coluna separada, nunca no lugar do dado bruto. Sem esse desconto, qualquer comparação entre modelos cobra do agente o defeito do nosso teste.

sprint	pacote	chamadas	culpa teste/plano	do modelo	decisão que descreve
004	P01	8	5	3	D-23: critério mandava .venv dentro do worktree (chamadas 1–6)
004	P04	10	5	5	D-23: idem (chamadas 1–5)

No total: **10 chamadas (20.0%)** foram gastas por defeito do nosso teste/plano e **2 (4.0%) por infraestrutura** (cota, crash). Sobram **38 chamadas (76.0%)** atribuíveis ao trabalho do modelo — esse é o único denominador honesto para comparar modelos.

Gráfico 2 — distribuição por número de tentativa



Percentual das chamadas por número de tentativa, com o modelo dominante de cada degrau

Tabela 2 — por degrau

tentativa	chamadas	% das chamadas	% acumulado	aprovadas	modelos usados
1ª	11	22.0%	22.0%	2	gpt-5.6-luna/low (11)
2ª	9	18.0%	40.0%	0	gpt-5.6-terra/low (9)
3ª	9	18.0%	58.0%	2	gpt-5.6-sol/low (9)
4ª	6	12.0%	70.0%	2	gpt-5.6-sol/medium (5), gpt-6-astra/low (1)
5ª	5	10.0%	80.0%	2	gpt-6-astra/low (4), gpt-5.6-sol/medium (1)
6ª	3	6.0%	86.0%	1	gpt-6-astra/low (2), gpt-5.6-luna/low (1)
7ª	2	4.0%	90.0%	0	gpt-5.6-luna/low (1), gpt-5.6-terra/low (1)
8ª	2	4.0%	94.0%	0	gpt-5.6-terra/low (1), gpt-5.6-sol/low (1)
9ª	2	4.0%	98.0%	1	gpt-5.6-sol/low (1), gpt-5.6-sol/medium (1)
10ª	1	2.0%	100.0%	1	gpt-6-astra/low (1)

A escada declarada × o que aconteceu

Escada de implementação no `models.yaml` (tentativa → modelo): 1ª `luna/low` · 2ª `terra/low` · 3ª `sol/low` · 4ª `sol/medium` · 5ª `astra/low` .

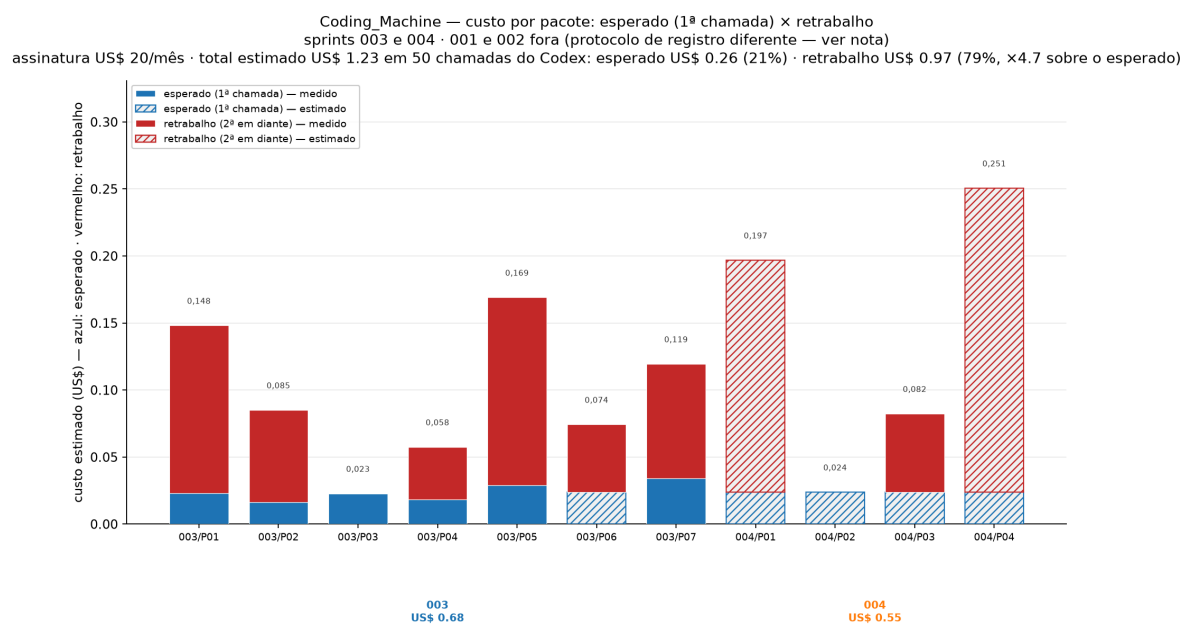
O que a base mostra é diferente em pontos importantes, e por motivos conhecidos:

1. **Contador da task ≠ número da chamada** (aviso 5) — a explicação principal. O contador reinicia no rearme/reabertura e volta ao degrau barato enquanto a chamada continua sendo numerada. Isso é **desejado**: nos rearmes de 28/09 a P01 e a P04 voltaram ao degrau 1 e subiram de novo — gastaram barato até acertar, em vez de continuar no topo.
2. **Falha de infraestrutura hoje ESCALONA** (aviso 5): a lista `classes_sem_escalonamento` existe na política, mas o orquestrador escolhe o modelo pelo contador e ignora o tier da decisão. A intenção declarada (“espera de cota não gasta degrau”) **não está fiada no código** — defeito registrado como pendência.

3. **Saturação depois da 5ª** (aviso 6): o mapa tem 5 entradas, então degraus ≥ 5 usam `astra/low`. No período o topo aparece em 8 das 50 chamadas.
4. **Buracos na numeração** (aviso 8) e **5 chamadas anteriores à padronização** da própria escada (aviso 7).

A pergunta de política que fica: escalonar por **número** (é o que existe) ou por **causa** — só escalar quando o teste do código falhar ou o revisor reprovar, e reiniciar no degrau barato quando a falha foi de infraestrutura/harness. A tabela 3 dá a medida de que lado pesa: os pacotes que aprovaram até a 3ª chamada mostram quanto trabalho se resolve sem sair do degrau mais barato.

Gráfico 3 — custo por pacote: esperado × retrabalho



Custo por pacote: azul = esperado (1ª chamada), vermelho = retrabalho; sólido = token medido pelo motor, hachurado = estimativa

Duas leituras na mesma barra. A **cor** diz se o dinheiro era **esperado** (a 1ª chamada do pacote, a tentativa de acertar de primeira) ou **retrabalho** (da 2ª em diante: cada chamada extra existe porque a anterior não passou). O **padrão** diz se aquele pedaço foi **medido** pelo motor (sólido) ou **estimado** pela régua do modelo (hachurado).

Esperado × retrabalho. Do total de US\$ 1,23, **US\$ 0,26 (21%)** era **esperado** e **US\$ 0,97 (79%)** é **retrabalho** — o mesmo backlog custaria **x4,7 menos** se todo pacote passasse de primeira. O múltiplo aqui é em **dinheiro**; o da Tabela 1 é em **contagem de reprovações**, e os dois não têm de coincidir (pacote que reprova muito com modelo barato pesa pouco em dólar, e vice-versa).

Pacotes que mais gastaram insistindo:

pacote	sprint	chamadas	esperado (US) retrabalho(US)	múltiplo	
P04	004	10	0,024	0,227	10,5x
P01	004	8	0,024	0,173	8,2x
P05	003	5	0,029	0,140	5,8x
P01	003	6	0,023	0,125	6,5x
P07	003	5	0,034	0,085	3,5x
P02	003	4	0,016	0,069	5,3x

Em chamadas: **11** foram a tentativa de acertar de primeira e **39** foram retrabalho.

O que é medido e o que é estimado. O motor só começou a gravar tokens em 28/09 (P-10, lendo o rodapé do Codex): **29 das 50 chamadas (58.0%)** têm token medido. A parte **sólida** da barra é essa medição; a **hachurada** são as chamadas sem medição, estimadas pela régua de tokens por chamada do modelo — a média **medida no próprio motor** para aquele modelo/esforço (5 modelo(s)) e, só para o que o motor nunca viu, a prévia do A/B de 28/09 (4 modelo(s)). Chamada sem régua nenhuma usa a **mediana** das medidas (52,386 tokens). Nada aqui vem de tabela de preço de terceiro.

O preço é o da sua assinatura: US\$ 20/mês = 4 blocos semanais de 100% → **US\$ 0,05 por ponto da janela semanal**; a régua medida é 118.096 tokens por ponto → **US\$ 0,423 por milhão de tokens**. Pela janela de 5h a leitura daria US\$ 0,37/Mtok (as duas estão no relatório de eficiência; a semanal é a que limita).

Total estimado: US\$ 1,23 para as 50 chamadas do Codex — sendo **US\$ 0,71 de token medido** e o resto estimativa. Os pacotes mais caros: **P04** (sprint 004) US\$ 0,251; **P01** (sprint 004) US\$ 0,197; **P05** (sprint 003) US\$ 0,169.

Objetivos dos pacotes (todas as sprints planejadas até agora)

Uma linha por pacote, com o objetivo como está no `dag.json` de cada sprint — lido do **plano**, não escrito à mão. É o mapa do que cada pacote do backlog pedia, para ler as tabelas acima sabendo o que estava sendo pedido em cada um.

sprint	pacote	objetivo (título do pacote no plano)
001	T01	Detectar agentes de código instalados
001	T02	Definir schemas de Sprint e de task
001	T03	Implementar store de estado em SQLite
001	T04	Implementar gerente de worktrees Git
001	T05	Implementar adaptador do Codex
001	T06	Implementar adaptador do Antigravity/AGY
001	T07	Implementar isolamento de execução (sandbox)
001	T08	Implementar runner de testes determinístico

sprint	pacote	objetivo (título do pacote no plano)
001	T09	Implementar revisor independente entre agentes
001	T10	Implementar logica de retry e recuperacao
001	T11	Implementar Fila de Acao Humana (HAQ)
001	T12	Implementar branch de integracao do Sprint
001	T13	Implementar geracao do relatorio do Sprint
001	T14	Criar projeto-fixture ponta a ponta
001	T15	Executar teste de aceitacao autonomo
002	P01	Criar autodev/planner.py com os tipos do plano e validacao de schema
002	P02	Criar autodev/plan_prompt.py com o prompt de planejamento
002	P03	Parsear a resposta do agente de forma tolerante
002	P04	Validar o plano contra o DAG real e contra criterio vago
002	P05	Escrever o sprint planejado em disco
002	P06	Invocar o agente em modo planejamento
002	P07	Adicionar o comando autodev plan na CLI
002	P08	Provar a aceitacao: do prompt ao sprint executavel
002	P09	Registrar a rastreabilidade do prompt ao plano
002	P10	Documentar o planejador e fechar a documentacao
003	P01	Detectar colisao de arquivo entre tasks do plano
003	P02	Barrar colisao na mesma onda no validador do motor
003	P03	Relatorio de impacto do plano em modulo proprio
003	P04	Comando autodev prever: impacto do plano antes de rodar
003	P05	Portao de aprovacao humana do plano
003	P06	Aceitacao ponta a ponta do portao do plano
003	P07	Documentar o portao do plano e o checklist derivado do D-16
004	P01	Restaurar a hierarquia de secoes do README
004	P02	Fazer o teste de documentacao isolar a secao do Planejador
004	P03	Tirar do README o que nao bate com o codigo

sprint	pacote	objetivo (título do pacote no plano)
004	P04	Comando de teste nos criterios tem de rodar nesta maquina

Arquivos gerados e proveniência

arquivo	o que é
REPORT-TENTATIVAS-CODEX.md	este relatório
report/histograma-tentativas.png	gráfico 1
report/distribuicao-tentativas.png	gráfico 2
report/custo-por-pacote.png	gráfico 3 (custo estimado: medido × estimado)
report/dados.json	agregação crua usada no texto e nos gráficos
report/gerar_relatorio_tentativas.py	gera os três a partir de <code>.autodev/state.db</code>
report/relatorio-tentativas.html / .pdf	HTML autocontido e PDF A4

Reproduzir:

```
cd ~/Code/Coding_Machine
~/hermes/cache/scratch/venv_report/bin/python report/gerar_relatorio_tentativas.py
pandoc REPORT-TENTATIVAS-CODEX.md -o report/relatorio-tentativas.html --standalone \
  --embed-resources --resource-path=.:report --css=report/estilo-relatorio.css \
  --metadata lang=pt-BR
chromium --headless=new --disable-gpu --no-sandbox --user-data-dir=/tmp/chrome-pdf \
  --virtual-time-budget=30000 --no-pdf-header-footer \
  --print-to-pdf="$PWD/report/relatorio-tentativas.pdf" \
  "file://$PWD/report/relatorio-tentativas.html"
```

O venv `~/hermes/cache/scratch/venv_report` existe só para o `matplotlib` (o venv do projeto não tem gráficos) e é descartável: `uv venv + uv pip install matplotlib`.